Advanced Side Channel Analysis Activity Proposal

ECE 576

By: Brayden Little and Cami Payne
Date: 10 / 15 / 26

1 Overview

1.1 Summary

This program will be designed to teach students the fundamentals of leakage assessment and profiling. It will be entirely run on software and be hardware independent. Students will interact with a game GUI with a built-in terminal that allows them to observe the traces from four simulated computer chips running AES-128, one of which will be leaky. They will run experiments, record the traces of the various chips, and use a t-test to determine which chip is leaky. Then, they will interact with a student-controlled copy of the chip to build a profile of its leakage by collecting traces under known plaintext and key values. Finally, they will observe traces from the leaky chip again and use their profile to recover one byte of its secret AES-128 key.

1.2 Learning Objectives

By the end of this workshop, participants will be able to:

1. Understand the importance of leakage assessment.
2. Understand how a profile is built and used in attacks.
3. Apply their understanding in a practical simulated scenario.

1.3 Subtopics to be Covered

Subtopics to be covered in this game/workshop are:

1. **Stage 0 — Leak Assessment:** Power consumption, noise, Hamming weight, AES, testing methodologies, Welch's t-test, TVLA, leakage interpretation.
2. **Stage 1 — Profiling:** Profiled threat models, traces, template attacks, building profiles.
3. **Stage 2 — Key Recovery:** Template matching, nearest-mean classification, key recovery.

2 Prerequisites

Target audience	Undergraduate Computer Science and Computer Engineering Students.
Assumed background	Ability to run python script to launch game GUI. Basic foundations of how chips are traced and what metrics are tracked (specifically power draw and electromagnetism readings). Some familiarity with linux-style command line input.

3 Structure

3.1 Format

The activity is a single pygame application. Students view traces and graphs on screen and work through a built-in terminal with a small command set (including help). No separate tools are needed.

3.2 Stage Breakdown

Stage 0 — LEAK ASSESSMENT

Objective. Students learn what leakage assessment is for. They will learn to figure out, before any attack is attempted, whether a device's power consumption depends on secret data. By the end of the stage they should be able to run a fixed-versus-random Welch's t-test, read the resulting curve against the TVLA threshold, and explain why a trace that merely looks busy or irregular is not evidence of leakage.

Activity. Students will interact with the game to observe the traces of the four chips under chosen conditions. They will use terminal commands to capture traces from a chosen chip, selecting a fixed or random plaintext input and the number of trials to collect, and to run the t-test on the collected data. Lastly, students will use the graphs they collect to infer which of the four chips is leaky.

Simulated leakage model. Each simulated chip models the first AES-128 round for one key byte. A trace is a fixed baseline (a simple sine wave) plus Gaussian noise (the amount of which will be chosen so that the leak is clearly discernible), generated in Python with NumPy. On the leaky chip, a short window of the trace also carries the Hamming weight of the S-box output, S-box(p XOR k), where p is the plaintext byte and k is the key byte. This is the standard Hamming weight model used in the ChipWhisperer tutorials. The other three chips have no key-dependent term: one is noise only, one is masked with a fresh random value for each trace, and one has a fixed pattern of peaks that is the same on every trace. Masking randomizes the value that leaks, so it has no fixed relationship to the key, and the fixed peaks are the same for fixed and random inputs, so they look busy but carry no data dependence. A fixed-versus-random t-test will therefore show a t-value above 4.5 on the leaky chip only, and only near the leak location. The leaky chip's identity and leak location are randomized each session.

Success criteria. Students identify the leaky chip and report its peak sample index, t-value, and traces per set.

Stage 1 — PROFILING

Objective. Students will learn how to build a profile of a device that they will then be able to use to extract a key byte for AES-128.

Activity. After deciding which chip is leaky in the first stage, students will build a profile of the simulated chip. The program will allow them to observe the traces from trials on a student-controlled copy of the chip. We simplify the activity to recovering a single key byte, which teaches the core concepts without making things take too long.

What is being profiled. The profile has nine classes, one for each possible Hamming weight of the S-box output. On the student-controlled copy, students choose the key and how many plaintexts to send, then run the profiling command. The command labels each trace with the Hamming weight of the S-box output and computes the mean trace value at the leak location found in Stage 0 for each class. The nine means form the profile, and students check that they rise steadily with Hamming weight.

Success criteria. Students submit the nine class means as a table or plot. The means should rise steadily with Hamming weight. The minimum traces per class will be set during development.

Stage 2 — KEY RECOVERY

Objective. Students will use the profile they developed to determine one byte of the secret key of the leaky chip.

Activity. Students will use the profile they built to attack the leaky chip. They will observe traces from the leaky chip, whose key is unknown, and compare them against the class means in their profile to infer which key byte value best explains the observed traces.

Attack procedure. Students capture a small number of traces from the leaky chip using random plaintexts and an unknown key. For each of the 256 possible key byte values, the attack command predicts the Hamming weight of the S-box output for every trace and measures how far the observed trace value is from the matching class mean in the profile. The guess with the smallest total distance is the recovered key byte. Because many byte values share the same Hamming weight, a single trace cannot identify the key byte, and the evidence builds up as more traces are collected.

Success criteria. Students recover the correct key byte, which the game will verify.

3.3 Timetable

Stage	Name	Time
0	LEAK ASSESSMENT	25 min
1	PROFILING	25 min
2	KEY RECOVERY	10 min
	Total	60 min

3.4 Scoring / Completion Criteria

Students submit a short write-up with the evidence listed in each stage's success criteria. The write-up must also explain, in the student's own words and citing their own numbers, why each non-leaky chip stays below the threshold using its peak t-value, and why a single attack trace cannot identify the key byte. Because chip identities, leak location, and key are randomized each session, answers can be checked against the game's recorded results.

4 Technical Requirements

4.1 Infrastructure

This activity will be designed to run on any OS that is capable of running python. Students will be provided instructions on how to start the program, including installing the required packages (numpy, scipy, and pygame).

4.2 Materials to Be Provided

- Repo containing all necessary scripts, code, and information
- README/pdf containing a description of the activity and instructions
- Challenge walkthrough that explains how the secret key can be determined. (Instructor only / distributed after activity is over)

5 AI Policy and Usage

5.1 Role in the game

AI will be used mainly to support the development of the GUI and interactive aspects of the game, as well as supporting the general development process and sanity checking work. LLMs will be incorporated in the game itself in the form of a AI helper bot in the game that students will be able to ask questions to. The bot will be given the full context of the game save for the answers that students will need to recover, and prompted to answer questions in a helpful way without giving away any real answers. The model will be hosted on a local lab server for sake of demonstration and to avoid API costs, but is easily extensible to make the bot function better and for larger groups via the many LLM APIs that exist. The game will automatically connect to the bot server, and in case of failure, not having the bot will not impede the flow of the game.

5.2 How will we prevent students from trivializing the game with LLMs?

Every required result comes from the student's own randomized session, so an LLM alone cannot produce it, and the write-up asks for explanations of those specific results. The built-in chatbot is never given the session's answers, and we will test this by trying to extract them before the activity is used. Students may use outside AI tools to learn concepts, since that does not replace the student actually doing the activity, but we encourage them to use the built-in chatbot.